

Branch: gh-pages genstats / inst / doc / 01_13_clustering.Rmd

jtleek 14 days ago big update

1 contributor

168 lines (119 sloc) 3.77 KB

RawBlameHistory

title	author	output	
Clustering	Jeff Leek	rmarkdown::html_document	vignette
		toc	
		true	%\VignetteIndexEntry{Clustering for genomics} %\VignetteEngine{knitr::rmarkdown} \usepackage[utf8]{inputenc}

Dependencies

This document depends on the following packages:

```
suppressPackageStartupMessages({
  library(devtools)
  library(Biobase)
  library(dendextend)
})

library(devtools)
library(Biobase)
library(dendextend)
```

To install these packages you can use the code (or if you are compiling the document, remove the `eval=FALSE` from the chunk.)

```
install.packages(c("devtools", "dendextend"))
source("http://www.bioconductor.org/biocLite.R")
biocLite(c("Biobase"))
```

General principles

- How do we define close?
- How do we group things?
- How do we visualize the grouping?
- How do we interpret the grouping?

Load some data

We will use this expression set to look at how we use plots and tables to check for different characteristics

```
con =url("http://bowtie-bio.sourceforge.net/recount/ExpressionSets/bodymap_eset.RData")
load(file=con)
```

```
close(con)
bm = bodymap.eset
pdata=pData(bm)
edata=as.data.frame(exprs(bm))
fdata = fData(bm)
ls()
```

Calculate distances between samples

- Most important step
 - Garbage in -> garbage out
- Distance or similarity
 - Continuous - euclidean distance
 - Continuous - correlation similarity
 - Binary - manhattan distance
- Pick a distance/similarity that makes sense for your problem

First we log transform and remove lowly expressed genes, then calculate Euclidean distance

```
edata = edata[rowMeans(edata) > 5000,]
edata = log2(edata + 1)

# By default calculates the distance between rows
dist1 = dist(t(edata))

## Look at distance matrix
colramp = colorRampPalette(c(3,"white",2))(9)
heatmap(as.matrix(dist1),col=colramp,Colv=NA,Rowv=NA)
```

Now cluster the samples

Here we use the distance we previously calculated to perform a hierarchical clustering and plot the dendrogram:

```
hclust1 = hclust(dist1)
plot(hclust1)
```

We can also force all of the leaves to terminate at the same spot

```
plot(hclust1,hang=-1)
```

We can also color the dendrogram either into a fixed number of groups

```
dend = as.dendrogram(hclust1)
dend = color_labels(hclust1,4,col=1:4)
plot(dend)
```

Or you can color them directly

```
labels_colors(dend) = c(rep(1,10),rep(2,9))
plot(dend)
```

This can get very fancy and we won't cover all the options here but for further ideas check out the [dendextend](#) package.

K-means clustering

Now we can perform k-means clustering. By default, the rows are clustered. You can either input the cluster means (often unknown) or the number of clusters (here I input 3 clusters)

```
kmeans1 = kmeans(edata,centers=3)
names(kmeans1)
```

Now we can look at the cluster centers

```
matplot(t(kmeans1$centers),col=1:3,type="l",lwd=3)
```

We can also look at how many belong to each cluster

```
table(kmeans1$cluster)
```

And cluster the data together and plot

```
heatmap(as.matrix(edata)[order(kmeans1$cluster),],col=colramp,Colv=NA,Rowv=NA)
```

Note that this is a random start so you get a different clustering if you run it again you get a different answer

```
kmeans2 = kmeans(edata,centers=3)
table(kmeans1$cluster,kmeans2$cluster)
```

Session information

Here is the session information

```
devtools::session_info()
```

It is also useful to compile the time the document was processed. This document was processed on: `r Sys.Date()` .

